Installation and setup of CASPER Toolflow for Red Pitaya Development

Using FPGA for data streaming and processing

Fazal A Kareem

Under the supervision of Dr. Prabu. T

Data Signal Processing Lab
Electronics Engineering Group
Raman Research Institute
Banglore.

Contents

1 Introduction . . . 2

2 Configuring Red Pitaya for Casper . . . 2

3 Installing and setting up Casper Toolflow for Red-Pitaya . . . 3

3.1 Installing Matlab . . . 4

3.2 Installing Xilinx Tools . . . 4

3.3 Installing casperfpga . . . 7

3.4 Installing the toolflow (mlib_devel) . . . 8

3.5 Configuring the Toolflow . . . 9

3.6 Miscellanious Fixes . . . 9

4 Configuration and Testing of Red Pitaya for Data processing and Streaming. . . . 12

4.1 Led-Blinker for testing connection . . . 12

4.2 Wideband Spectrometer . . . 14

5 Conclusion and Discussion . . . 16

6 Appendix . . . 17

6.1 Python file to run the spectrometer to obtain plots . . . 17

6.2 Python file to run the spectrometer and save the data to bitfiles. . . . 19

6.3 Python script to test the output from stream_dbg.py . . . 21

1 Introduction

If you find more errors or fixes, please add to this overleaf file to keep it updated. Link

2 Configuring Red Pitaya for Casper

This is the most important page for you to follow in this.[1][2]

I am logging all the steps I did here.

I installed Red Pitaya OS on a new SD card, and these are the steps I followed.

- Download the stable 1.04 Red Pitaya OS from here[3] You can download the latest stable version under the name Latest Stable 125-14/10 under 1.3.1.3. Latest and Beta versions 1.04 OS.

  I burned the OS into the SD card using Applepie as I was using a MacBook. You can follow the steps to do this using Ubuntu exactly as it is given on the above website.

  Once you have the OS ready it is time for us to configure the RP to talk to casper through KATCP.

Running the script on a preloaded RP SD Card.

- Insert your SD card and boot the RP, ensuring that the RP is connected to your Ethernet network
- SSH into the RP using the hostname printed on the Ethernet port of the board (default user:root password:root)
- Run the following script on the RP:

```
# make sure /etc/hosts exists
touch /etc/hosts
# make sure localhost is in /etc/hosts (required by
   tcpborphserver3)
grep -q localhost /etc/hosts || echo "127.0.0.1 localhost"
   >> /etc/hosts
# install git
apt-get install git
# clone katcp
git clone https://github.com/ska-sa/katcp.git
# build katcp
cd katcp
```

```
make all
# copy executables to /bin
cp cmd/kcpcmd /bin/
cp fpg/kcpfpg /bin/
cp tcpborphserver3/tcpborphserver3 /bin/




# create startup service file
# run the next 11 lines together.
echo "Description=TCPBorphServer allows programming and
   communication with the FPGA
Wants=network.target
After=syslog.target network-online.target
[Service]
Type=simple
ExecStart=/bin/tcpborphserver3
Restart=on-failure
RestartSec=10
KillMode=process
[Install]
WantedBy=multi-user.target" > /etc/systemd/system/
   tcpborphserver.service


# reload services
systemctl daemon-reload
# enable the service
systemctl enable tcpborphserver
# start the service
systemctl start tcpborphserver
# check the status of your service
systemctl status tcpborphserver
```

- This will install git on the RP, clone and build tcpborphserver (a server designed to control and speak to CASPER hardware using KATCP), and then set it to run on startup.
- Your RP is now casperized and you can communicate with it via casperfpga.

3 Installing and setting up Casper Toolflow for Red-Pitaya

This report is a side note to following the exact installation steps on casper[4] toolflow installation guide. We tested Ubuntu version 20.04 with Matlab version 2021a, Xilinx version Vivado 2021.1,

and mlib_devel branch m2021a.

Ubuntu 20.04 LTS comes with gcc .7x to .9x. This is not compatible with Xilinx, and hence, we will either have to install gcc .6x or make symbolic links as below. I chose the second method.

```
sudo ln -s /usr/include/asm-generic /usr/include/asm
sudo ln -s /usr/include/x86_64-linux-gnu/sys /usr/include/sys
sudo ln -s /usr/include/x86_64-linux-gnu/bits /usr/include/bits
sudo ln -s /usr/include/x86_64-linux-gnu/gnu /usr/include/gnu
```

Listing 1: gcc .6x setup

3.1 Installing Matlab

1. Download the Matlab installer from here[5]. Select the 2021a installer and download it for Linux
2. Run the following command in the terminal:

```
cd /path/to/matlab/download/matlab_R2021a_glnxa64.zip
mkdir matlab_R2021a
unzip matlab_R2021a_glnxa64.zip -d matlab_R2021a
cd matlab_R2021a
sudo ./install
```

3. The above will launch the MathWorks product installer, and you will be prompted to sign into your MathWorks account. Sign in, accept the licensing agreement, link your MATLAB license, and select your installation path in the prompts that follow.

   Tip: Use the keyboard to move around the installer 1 instead of the mouse. Using a mouse is causing a bug and tends to stop the installation in between. (space to select and tab to move selection).

4. Select these toolboxes to install 1:
5. Continue through the installation as usual.

3.2 Installing Xilinx Tools

Now we will install Xilinx Vivado and Vitis Model Composer. Go to this page[6].

1. Select Version 2021.1 from Vivado Archives.

   In the above screengrab 2, I have selected the Unified offline installer. This will help us install and reinstall easily if there are any issues during installation. This is a big file so the download can take some time.

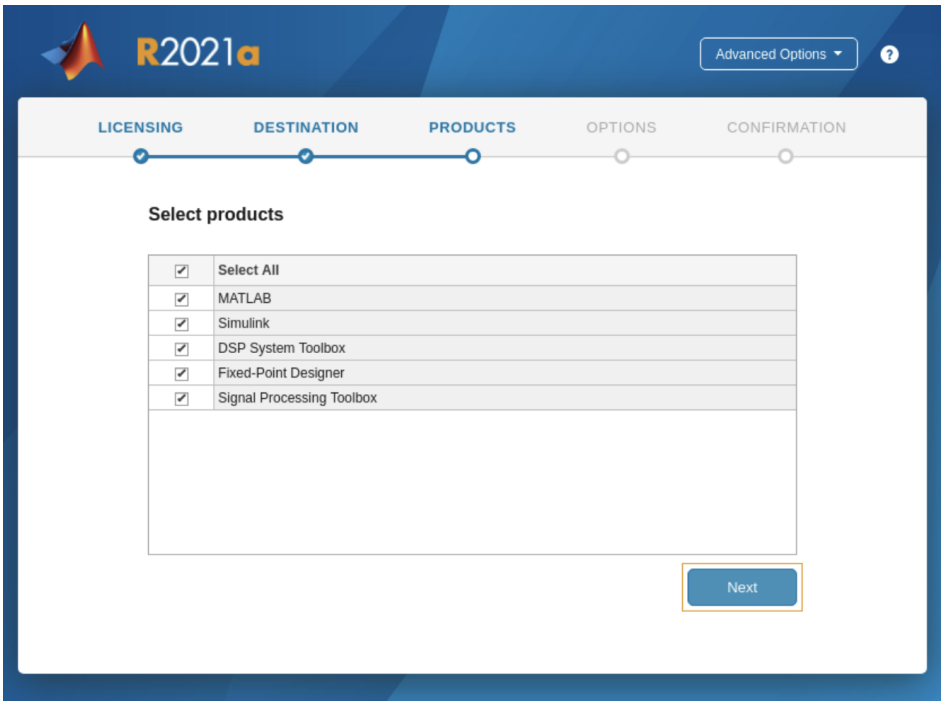


Figure 1: Matlab installation window



Figure 2: Vivado Downloader. Download the complete package of about 60GB instead of the online installer.

2. The installation will hang towards the end for any installation unless we take care of some library files. Run these commands on your terminal before starting the installation to avoid this.

```
### AIE Tools prerequisite libraries
apt-get update | tee -a $logFile
apt-get install -y libc6-dev-i386 net-tools | tee -a
   $logFile
apt-get install -y graphviz | tee -a $logFile
apt-get install -y make | tee -a $logFile
```

```
6
7  ### Vitis Tools prerequisite libraries
8  apt-get install -y unzip | tee -a $logFile
9  apt-get install -y g++ | tee -a $logFile
10 apt-get install -y libtinfo5 | tee -a $logFile
11 apt-get install -y xvfb | tee -a $logFile
12 apt-get install -y git | tee -a $logFile
13 apt-get install -y libncursesw5 | tee -a $logFile
14 apt-get install -y libc6-dev-i386 | tee -a $logFile
```

3. Unzip the downloaded file.

4. Go to Xilinx/2021.1/

```
1 /Xilinx/2021.1$ sudo ./xsetup
2 This is a fresh install.
3 ......
4 ....
```

   Now select vivado from the installer and install all the defaults. Check all the permissions and certificates.

5. You can use the default /tools/Xilinx folder as the installation folder. If it says that this folder doesn't exist. Then, sudo create the folder and restart the installation.

6. Click on finish and go for a cup of coffee...

7. If you are lucky enough, you'll get this message 3 on your first try itself.

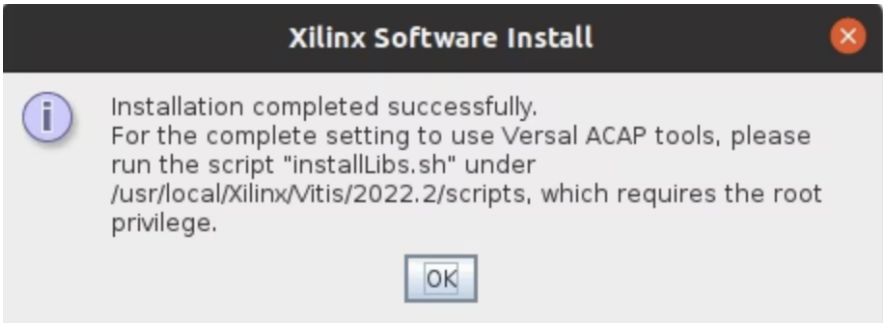


Figure 3: Installation completed message from Xilinx

8. You can also install the USB cable drivers, this is not mandatory

```
1 cd /usr/local/Xilinx/Vivado/2022.2/data/xicom/cable_drivers
     /lin64/install_script/install_drivers
2 sudo ./install_drivers
```

9. Now that you have the time, please run this command so that you don't run into other issues with matlab and casper later on:

3.3 Installing casperfpga

This section explains how to install casperfpga, a Python library used to interact with CASPER hardware. The casperfpga library currently works fully with Python 3.8.

Compiling for supported platforms If you don't have these already you can probably install them in Ubuntu environments by opening a terminal and running the command

```
apt-get install python3.8 python3-pip
```

casper recommends using a virtual environment to separate the version of Python and its libraries the toolflow uses from the rest of your system.

```
# install the python3-venv package
sudo apt install python3-venv
# change directory to where you want the virtual
   environment to live
cd /home/user/work
# create a Python 3 virtual environment
python3.8 -m venv cfpga_venv
# to activate the virtual environment:
source cfpga_venv/bin/activate
# to deactivate the virtual environment:
deactivate
```

Clone the casperfpga git repo after activating this venv.

```
source ./cfpga_venv/bin/activate
git clone https://github.com/casper-astro/casperfpga
cd casperfpga/
git checkout py38
pip install -r requirements.txt
pip install .
```

If you see any progska errors during casperfpga installation, please run this command.

```
sudo apt-get --reinstall install libc6 libc6-dev
```

This pip install tends to throw a lot of dependency errors. If you see numpy, katcp, tornado, or circus errors, modify the requirements file as:

```
katcp
numpy == 1.20
circus == 0.16.0
tornado == 4.5
```

If jasper throw any numpy attribute errors later on, know that unless you solve these dependency errors,you won't be able to run the compiler.

Testing that the installation worked

To check that casperfpga has been installed correctly open an ipython session and import casperfpga.

**To avoid errors, move out of your cloned casperfpga repository directory before doing this test.

** 'casperfpga.__version__' will output the build and githash version of your casperfpga library.

```
cd ..
ipython
```

```
In [1]: import casperfpga
In [2]: casperfpga.__version__
```

If you receive any errors during this step please contact anyone on the CASPER Mailing List, or check the Mailing List Archive to see if your issue has been resolved already, I got a lot of advice from these archive chats.

3.4 Installing the toolflow (mlib_devel)

Clone the toolflow from the mlib_devel git repository.

```
git clone -b m2021a https://github.com/casper-astro/mlib_devel
   --depth=1
```

Next, move into the mlib_devel repository you have just created, activate your virtual environment (if using one) and download any Python dependencies you need by installing the requirements.txt file. The downloaded dependencies will be installed within the virtual environment separate from the rest of your system.

```
cd mlib_devel
source ./cfpga_venv/bin/activate
pip3 install -r requirements.txt
```

If this is showing errors, you might have to run pip command as sudo. (Check this once with prabu)

3.5 Configuring the Toolflow

If you have successully installed the toolflow and its dependencies, it is now time to configure the flow. The toolflow needs to know where dependencies like MATLAB and Xilinx tools have been installed and this is user defined through the startsg.local file.

The startsg script

A startup script – startsg – is provided as part of the toolflow repository. This script can be used in two ways:

1. If executed (i.e. /path/to/mlib_devel/startsg): start MATLAB with the correctly defined library paths.
2. If sourced (i.e. source /path/to/mlib_devel/startsg): configure software paths without starting MATLAB.

. The former method is what you should do if you want to start a Simulink design or open an existing one. The latter method is useful if you want to run parts of the toolflow outside of MATLAB (eg. exec_flow.py) or run Xilinx tools (eg. vivado) directly from the command line.

Configure the startsg.local file according to this Casper help page. . Give the path to your python virtual environment inside the startsg.local file. Now before you run the startsg command, we need to remove some of the file conflicts so that Matlab and Xilinx are not conflicting. I learned this the hard way, you are also welcome to try out the current setup before running these commands to find out what exactly are the issues mitigated by these commands.

3.6 Miscellanious Fixes

- If there are any wheel or build errors. Please make sure that you install the wheel and build, and binutils dependencies using pip.[7]
- Some issue is caused by library dependency differences between what Ubuntu base uses and what the Xilinx Model Composer and MATLAB Simulink want to use. Particularly the various libgmp libraries used.

  To side step this issue requires altering your Model Composer installation directory. I have been successful in making these adjustments. But, altering the installation may have other consequences and not generally recommended. That being said I have not as of yet had any compatibility or seen any failures result because of it.

  Assuming you have Xilinx installations at /tools/Xilinx (or change this path for where your installation is)

```
cd /tools/Xilinx/Model_Composer/2021.1/lib/lnx64.o/Ubuntu
mkdir exclude
mv libgmp.so* exclude/

cd /tools/Xilinx/Vivado/2021.1/lib/lnx64.o/Ubuntu
mkdir exclude
mv libgmp.so* exclude/

cd /tools/Xilinx/Vivado/2021.1/lib/lnx64.o
mkdir exclude
mv libgmp.so* exclude/
```

The libgmp that match the wild card will be moved to the exclude directory and not searchable when Model Composer and Simulink are load up. If you have any issues the process is reversed by placing the files in the exclude/ directory back into their previous locations.

- When running a simulation in my Ubuntu 20.04 system I got "Error: no such instruction: 'endbr64'".

  I spent a lot of time on this including making links to /usr/include without success. Then I tried

```
!gcc /tmp/hello.c
```

  /tmp/cc2Od0Ug.s: Assembler messages: /tmp/cc2Od0Ug.s:12: Error: no such instruction: 'endbr64'

```
!which as
```

  /srv/data/tools/Xilinx/Vivado/2021.1/tps/lnx64/binutils-2.26/bin/as in the Matlab console. Aha! The problem is obvious because the version of the system as is 2.34.

```
cd  /tools/Xilinx/Vivado/2021.1/tps/lnx64/binutils-2.26 I
mv bin bin.bak

## and compiling and simulating now works!
```

- You will have to update the Xilinx license before running any compilation. Get your license from the license manager inside your Xilinx account. Read more about this on the Xilinx website. For this, you have to run the VLM script, which is buried inside the Xilinx installation. Here is the location of the script.

```
/tools/Xilinx/Vivado/2021.1/bin/vlm
```

- **Cannot write to register error**[8] This can occur due to many reasons. To begin with, make sure that the red pitaya OS is not the unified 2.x version. Instead, select the 1.04x versions of the Red Pitaya OS to reduce dependency errors.

- Error evaluating 'OpenFcn' callback of Xilinx Adder/Subtracter Block block (mask) 'IIR_DirectForm_3pole_1/Ac
  Callback string is 'xlOpenGui(gcbh, 'addsub_gui.xml', @addsubenablement, -1)' Error using xlNMIProxy Timed out waiting for a response from GUI to: (2.0001) buildGUI DISPLAY ENV = ":0" timeout value = 180.0014

  This is a Qt4 package error.

  This error is caused by sysgensockgui failing to load. The non-loading is likely caused by missing dependencies. One these are resolved, sysgensockgui should operate as intended. Missing dependencies can be checked the following way,

```
root@r*****a:/tools/Xilinx/Vivado/2021.1/bin# ldd unwrapped
   /lnx64.o/sysgensockgui

[out]
linux-vdso.so.1 (0x00007ffedf51e000)
libQtCore.so.4 => not found
libQtGui.so.4 => not found
libQtNetwork.so.4 => not found
libQtXml.so.4 => not found
librdi_itlib.so => not found
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6 (0
   x00007fddc2160000)
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0
   x00007fddc200f000)
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0
   x00007fddc1ff4000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0
   x00007fddc1e02000)
/lib64/ld-linux-x86-64.so.2 (0x00007fddc2367000)

## "sysgensockgui" needs the entire framework of Qt 4.8.x
   to run correctly. If you have Qt 4 installed in your
   system, those should be taken care of for you.

##The four libraries libQtCore.so.4, libQtGui.so.4,
   libQtNetwork.so.4, and libQtXml.so.4 is our version of
   the Qt 4 libraries. It will be overloaded when you launch
    the SysGen blocks within the Qt 4 framework.
```

  The solution for this is to separately install these dependencies.

```
sudo add-apt-repository ppa:rock-core/qt4
sudo apt-get update
sudo apt-get install libqtcore4
sudo apt-get install libqtgui4
```

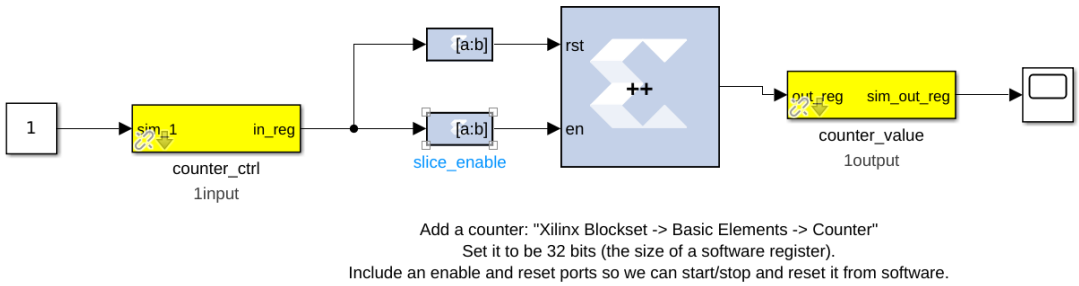


Figure 4: Simulink design for LED blinker

4 Configuration and Testing of Red Pitaya for Data processing and Streaming.

Now that we have everything setup, we can get into configuring the fpga for our data. RP has a 125MSPS clock but a 50MHz adc. So we will be working with data in the range of 0Hz - 50 MHz. We aim to build a spectrometer setup that can continuously query for data and store this in a file on the computer. To begin with let us make sure that we have a working setup. We will build an LED Blinker FPG file using Simulink and compile it using jasper. Then we will upload it to the RP and program the FPGA using casper toolflow.

All the codes used are uploaded here: Github. These files were obtained from the tutorial part of the casper documentation. Try building the design from scratch to get a feel for the casper toolflow.

4.1 Led-Blinker for testing connection

To begin with, this screenshot is from the led-blinker test file inside the github repo. Follow the casper toolflow tutorial here to create a design from scratch.

In the design, Figure 4, The yellow blocks are casper blocks. Here both the yellow blocks are registers. These registers are used to initiate the counter and the other one for the scope. The grey coloured blocks are the xilinx blockset.

Compiling the simulink design

You can run debugs and simulations using simulink. The scope should be giving an output like this Figure 5:

Once you have this design, go to the matlab window and run the command to compile the design:

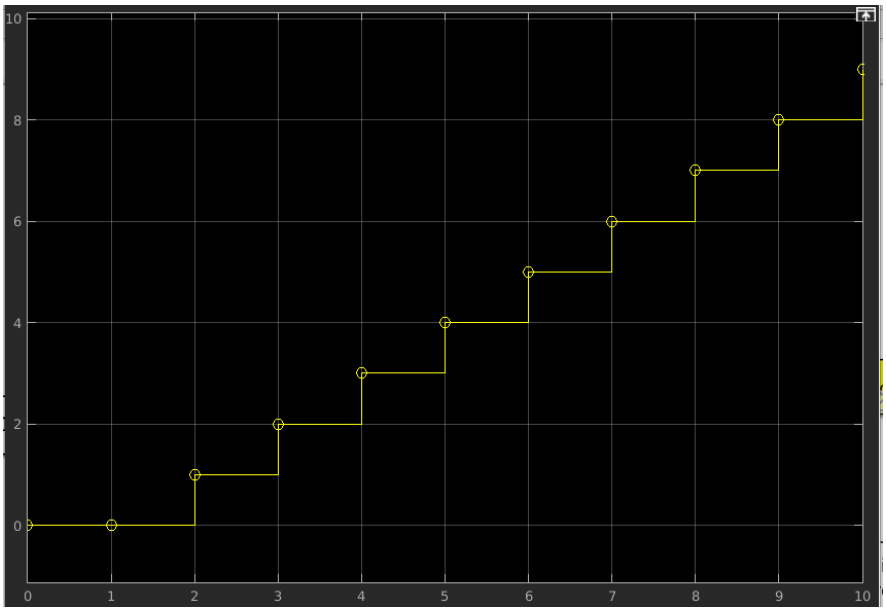


Figure 5: Simulation output of LED blinker from scopes

```
jasper
```

This will run the complete build process, which consists of two stages. The first involving Xilinx's System Generator, which compiles any Xilinx blocks in your Simulink design to a circuit which can be implemented on your FPGA. While System Generator is running, you should see the following window pop up as in Figure 6:

After this, the physical implementation of your design will happen through vivado and finally the toolflow will create the final output fpg file that you will use to program your FPGA. This will be inside the output folder inside the directory of your .slx file.

Connecting and programming the red-pitaya

```
ipython
import casperfpga
fpga = casperfpga.CasperFpga('red_pitaya_hostname or ip_address')
fpga.upload_to_ram_and_program('<your_fpgfile.fpg>')

## This will start the led blinker. If you are using the slx/fpg file from the tutorial, there will also be an adder in the design. To test this

fpga.write_int('a',10)
fpga.write_int('b',20)
```

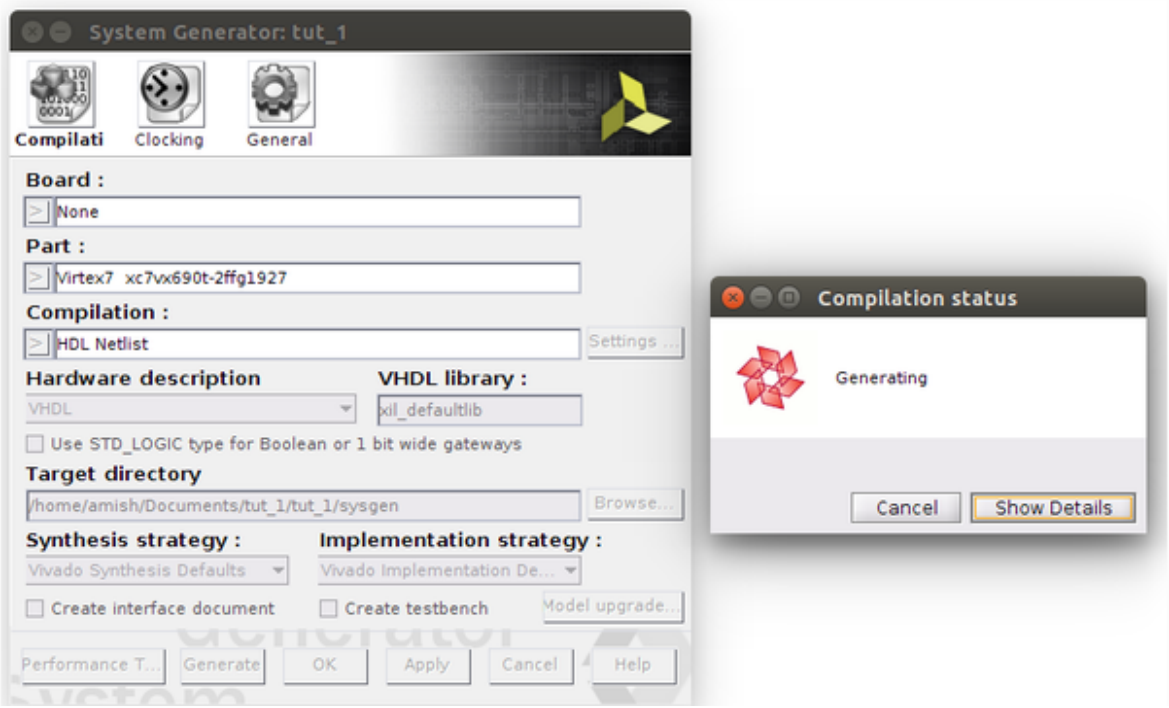


Figure 6: Pop up window during compilation using jasper

```
fpga.read_int('sum_a_b')

## If you find any issues about writing or reading from fpga,
   please refer to the previous misc fixes. And also make sure
   that you are using the 1.04 version of Red-Pitaya OS.
```

4.2 Wideband Spectrometer

A spectrometer is something that takes a signal in the time domain and converts it to the frequency domain. In digital systems, this is generally achieved by utilising the FFT (Fast Fourier Transform) algorithm.

For this project, we aimed to build a spectrometer that can dump the data to a file on the computer. We tested the feasilibility of using RP as a reciever backend using the already provided FPG file in the casper tutorials for Red Pitaya. The aim is to reduce the data size by power accumulating the input data after an FFT algorithm. All of the files used for this section are uploaded in this github repo.[9]

Since we have used the already existing design file, there is no issue in using the precompiled fpg file in the github repo. To test the design file, I ran debugs and simulations on it and also compiled the design to make sure there are no issues in the design. Now we will focus on the

data acquision part of the spectrometer.

For a regular data acquision we used the test_spec.py file. This code is provided in the appendix 2 and also available on the github repo.

To run this file I use the command:

```
python test_spec.py -f test_spec.fpg -r <Red-Pitaya IP or
   hostname> -a <accumulation length>
```

This will output 4 plots, 2 each for each channel on the RP IO. Given an input signal with a bandpass between 20MHz and 40 MHz as in Figure 7

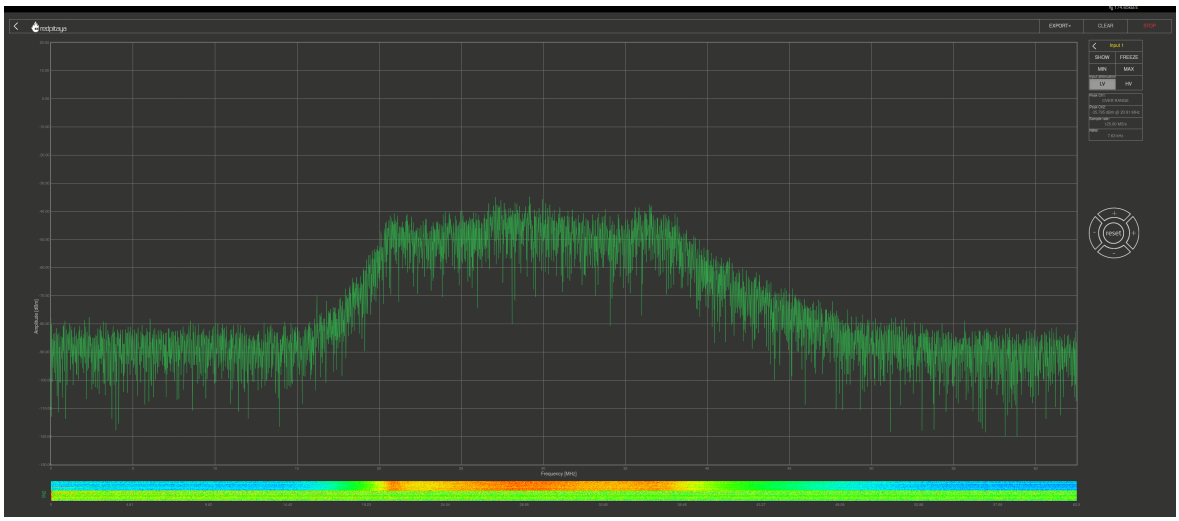


Figure 7: Input signal for a test case with bandpass between 20MHz and 40MHz

We will get these outputs from the code as in Figure 8:

To extend this further and write the data to a bit file, I modified the code so that we get a stream like output. This code is given in the github repo as stream_dbg.py (listing 3 and also provided in the appendix.

This script outputs the data obtained from the bram as *.npy files. The script also provides a runtime output of the data acquisition. Here the most important part of the design is the snap block (Figure 9)and we must take a deeper look into this block to figure out continuous streaming of the data.

For more details about the snap block, please look at the block documentation[10] on the casper docs. There are plenty of switches and bolts we can alter in the simulink design once we go into the design.

The script to test the output from stream_dbg.py file is given in the github repo with the name testing.py (listing 4. This is also provided in the appendix. The numpy bit files will have data from n number of cycles, each cycle containing 4096 samples of Power. Each 4096 samples

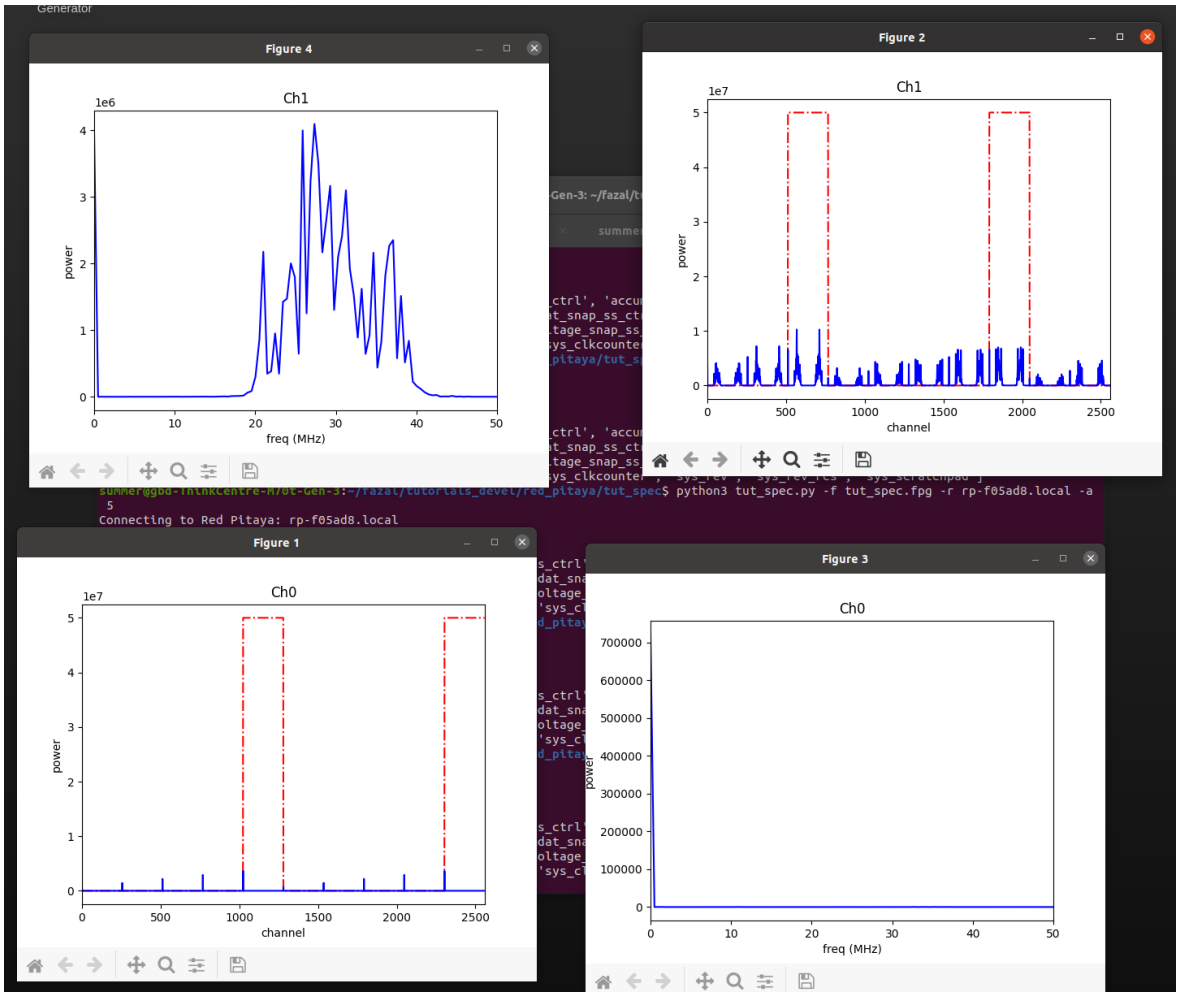


Figure 8: The 4 plots ashows the 2 channels on the RP IO. As evident Ch1 was connected to the input and the first plot shows the spectrum of this input. The second plot shows the accumulated power. Since the Accumulation length was set to 5, we can see that after every 5 accumulation, the accumulator resets to zero and starts a new accumulation.

will be divided into 16/acc_len sets of fft outputs. Try printing this out to have a feel for the file structure. As of now I have used a dictionary for ease of data capture, once we reach a high enough time criticality of this data structure, we can and should shift to numpy arrays instead.

5 Conclusion and Discussion

This work outlines the setup and use of RP for data processing and streaming using the casper toolflow. Although we didn't get to a streaming level, we were able to setup the insfrastructure required to acquire data in semi real time from the RP board. We were able to reach a query runtime of 2.5 ms with an accumulation length of 1400, which would be enough to get continuous

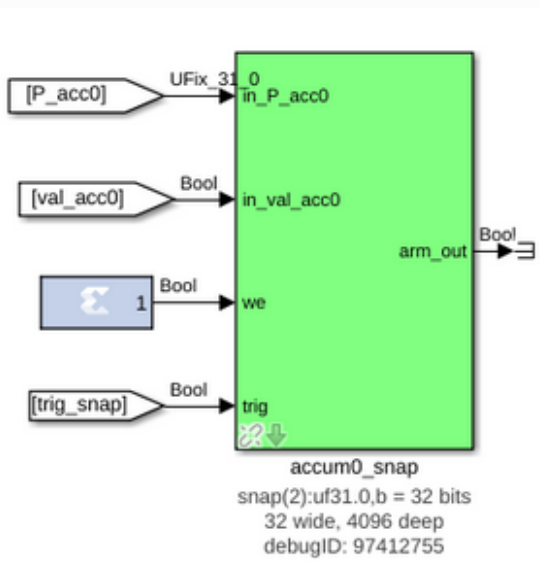


Figure 9: The snap block is a masked block of multiple other internal components. There is a cyclic arming (circular capture) option in the snap block which should also be looked at.

data streams from the board. I was not able to implement this continuous stream as this would probably require some modifications to the design file.

Circular capture and use of the offset and trigger of the snap block should be looked at. And also the use and depth of the bram should also be considered. The bram depth is taken as 4096 bits in the current design and once bram is completely filled, there is no overwriting happening. This will lead to loss of data during streaming.

6 Appendix

6.1 Python file to run the spectrometer to obtain plots

This is the test_spec.py code used for regular spectrometer usage.

```
import casperfpga
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import argparse
import time


parser = argparse.ArgumentParser()
```

```
parser.add_argument("-f", "--file", help="fpg file to upload to
    red-pitaya")
parser.add_argument("-r", "--redpitaya", help="Red-Pitaya
   hostname or  IP address")
parser.add_argument("-a", "--accums", help="Number of
   accumulations",default=4)
args = parser.parse_args()


red_pitaya = args.redpitaya
print("Connecting to Red Pitaya: {0}".format(red_pitaya))
fpga=casperfpga.CasperFpga(red_pitaya)

file_fpg=args.file
print("Uploading: {0}".format(file_fpg))
fpga.upload_to_ram_and_program(file_fpg)

fft_len=256
acc_len=int(args.accums)
snap_cyc=10
print("These are the devices in your design ...")
print(fpga.listdev())

fpga.write_int('acc_len',acc_len)
fpga.write_int('snap_gap',snap_cyc)
fpga.write_int('reg_cntrl',1)

time.sleep(5)

fpga.snapshots.accum0_snap_ss.arm()
spec0=fpga.snapshots.accum0_snap_ss.read(arm=False)['data']

fpga.snapshots.accum1_snap_ss.arm()
spec1=fpga.snapshots.accum1_snap_ss.read(arm=False)['data']

fpga.snapshots.accumdat_snap_ss.arm()
spec_dat=fpga.snapshots.accumdat_snap_ss.read(arm=False)['data'
   ]


fig0, ax0= plt.subplots()
ax0.plot(np.array(spec0['val_acc0'][0:2*acc_len*fft_len]).
   astype(float)*5e7,'r-.')
ax0.plot(spec0['P_acc0'][0:2*acc_len*fft_len],'b-')
ax0.set(xlabel='channel',ylabel='power',title='Ch0')
```

```
ax0.set_xlim(0,2*acc_len*fft_len)

fig1, ax1= plt.subplots()
ax1.plot(np.array(spec1['val_acc1'][0:2*acc_len*fft_len]).
    astype(float)*5e7,'r-.')
ax1.plot(spec1['P_acc1'][0:2*acc_len*fft_len],'b-')
ax1.set(xlabel='channel',ylabel='power',title='Ch1')
ax1.set_xlim(0,2*acc_len*fft_len)

fig2, ax2= plt.subplots()
valid = np.array(spec0['val_acc0'][0:2*acc_len*fft_len]).astype
    (bool)
spectrum0 = np.array(spec0['P_acc0'][0:2*acc_len*fft_len])
spectrum0 = np.fft.fftshift(spectrum0[:256])
ax2.plot(np.linspace(-256/2,256/2-1,256)*125/256,spectrum0
    [:256].astype(float),'b-')
ax2.set(xlabel='freq (MHz)',ylabel='power',title='Ch0')
ax2.set_xlim(0,50)

fig3, ax3= plt.subplots()
valid = np.array(spec1['val_acc1'][0:2*acc_len*fft_len]).astype
    (bool)
spectrum1 = np.array(spec1['P_acc1'][0:2*acc_len*fft_len])
spectrum1 = np.fft.fftshift(spectrum1[:256])
ax3.plot(np.linspace(-256/2,256/2-1,256)*125/256,spectrum1
    [:256].astype(float),'b-')
ax3.set(xlabel='freq (MHz)',ylabel='power',title='Ch1')
ax3.set_xlim(0,50)
plt.show()
```

Listing 2: test_spec.py script for plotting

6.2 Python file to run the spectrometer and save the data to bitfiles.

This is the stream_dbg.py file that I created and is given in the github repo. This script configures the red pitaya using the fpg file input. And then query the data output from the bram and save it in specX.npy files. These files can be later called to test using the testing.py script. This python script also gives the time taken for obtaining the data output.

```
import casperfpga
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import argparse
import time
```

```
import pickle


parser = argparse.ArgumentParser()
parser.add_argument("-f", "--file", help="fpg file to upload to
    red-pitaya")
parser.add_argument("-r", "--redpitaya", help="Red-Pitaya
   hostname or  IP address")
parser.add_argument("-a", "--accums", help="Number of
   accumulations",default=4)
parser.add_argument("-c", "--cycle", help="Number of cycles",
   default=5)
#parser.add_argument("--trig", default='False')
#parser.add_argument("--man_valid", default = 'False')
#parser.add_argument("--offset", default = -1)
#parser.add_argument("--circ_cap", default='False')
args = parser.parse_args()


red_pitaya = args.redpitaya
print("Connecting to Red Pitaya: {0}".format(red_pitaya))
fpga=casperfpga.CasperFpga(red_pitaya)

file_fpg=args.file
print("Uploading: {0}".format(file_fpg))
fpga.upload_to_ram_and_program(file_fpg)

fft_len=256
acc_len=int(args.accums)
snap_cyc=0
print("These are the devices in your design ...")
print(fpga.listdev())

fpga.write_int('acc_len',acc_len)
fpga.write_int('snap_gap',snap_cyc)
fpga.write_int('reg_cntrl',1)

accum_cyc=int(args.cycle)
all_spec0 = {}
all_spec1 = {}
all_spec_dat = {}

#fpga.snapshots.accum0_snap_ss.arm(man_trig=args.trig,
   man_valid=args.man_valid, offset = int(args.offset),
   circular_capture = args.circ_cap)
```

```
fpga.snapshots.accum0_snap_ss.arm()
start = time.time()
# Open the files for writing before the loop
with open('spec0.npy', 'ab') as spec0_file, open('spec1.npy', 'ab') as spec1_file, open('spec_dat.npy', 'ab') as spec_dat_file:
    for i in range(accum_cyc):
        print('cycle = ', i+1)
        spec0 = fpga.snapshots.accum0_snap_ss.read()['data']

        #fpga.snapshots.accum1_snap_ss.arm()
        #spec1 = fpga.snapshots.accum1_snap_ss.read(arm=False)['data']

        #fpga.snapshots.accumdat_snap_ss.arm()
        #spec_dat = fpga.snapshots.accumdat_snap_ss.read(arm=False)['data']

        # Add the data to the dictionaries
        all_spec0[f'cycle_{i+1}'] = spec0
        #all_spec1[f'cycle_{i+1}'] = spec1
        #all_spec_dat[f'cycle_{i+1}'] = spec_dat

    # Save the dictionaries to .npy files
    np.save('spec0.npy', all_spec0)
    #np.save('spec1.npy', all_spec1)
    #np.save('spec_dat.npy', all_spec_dat)


# Files will be automatically closed after the with block

end = time.time()
print(start, end, 'runtime: ', (end-start), 'seconds')
```

Listing 3: The stream_dbg script I created to acquire the data to a bitfile

We tried to use the other trigger and offset parameters of the arm function, but this was not working since the design file didn't activate these features before compiling.

6.3 Python script to test the output from stream_dbg.py

This is the testing.py script given in the github repo.

```

```

```
## This will take 2 arguments, first one the name of the numpy
   bit file to test, and second one the cycle number of the
   output that you want to test.

import numpy as np
import matplotlib.pyplot as plt
import sys

# Arguments passed
print("\nName of datafile:", sys.argv[1])

datafile = sys.argv[1]
cycle = sys.argv[2]
#spec0 = np.load(datafile, allow_pickle=True)
spec0 = np.load('spec0.npy', allow_pickle=True)
P_acc0 = spec0.item().get(cycle).get('P_acc0')

fig,ax = plt.subplots(4,4)
for idx in range(16):
    x1 = idx *256
    x2 = 256 * (idx + 1)
    #print(x1,x2)
    ax[int(idx/4),np.mod(idx,4)].plot(P_acc0[x1:x2])
    ax[int(idx/4),np.mod(idx,4)].set_ylim(0,1.3e7)
    print(int(idx/4),np.mod(idx,4))

print('Number of cycles in file :',spec0.item().keys())
print(len(P_acc0))
print('Total data length = ', len(P_acc0) * len(spec0.item().
   keys()))
#plt.plot(P_acc0)
plt.show()
```

Listing 4: This is the testing.py script to plot and test the output files from the stream_dbg script

Bibliography

[1] *Guide to Setting Up Your New Red Pitaya — CASPER Tutorials 0.1 documentation*. [Online; accessed 25. Oct. 2023]. Mar. 2023. URL: https://casper-toolflow.readthedocs.io/projects/tutorials/en/latest/tutorials/redpitaya/red_pitaya_setup.html.

[2] *Installing Xilinx Vivado on Ubuntu 20 & 22*. [Online; accessed 25. Oct. 2023]. Oct. 2023. URL: https://community.element14.com/technologies/fpga-group/b/blog/posts/installing-xilinx-vivado-on-ubuntu.

[3] *1.3. Prepare SD card — Red Pitaya 2.00 documentation*. [Online; accessed 25. Oct. 2023]. Oct. 2023. URL: https://redpitaya.readthedocs.io/en/latest/quickStart/SDcard/SDcard.html.

[4] *Installing the Toolflow — CASPER Toolflow 0.1 documentation*. [Online; accessed 25. Oct. 2023]. Mar. 2023. URL: https://casper-toolflow.readthedocs.io/en/latest/src/Installing-the-Toolflow.html.

[5] *MathWorks Account Sign In*. [Online; accessed 25. Oct. 2023]. Oct. 2023. URL: https://in.mathworks.com/login?uri=%2Fdownloads%2Fweb_downloads.

[6] *Downloads*. [Online; accessed 25. Oct. 2023]. Oct. 2023. URL: https://www.xilinx.com/support/download.html.

[7] *Error: invalid command 'bdist_wheel' in Python [Solved] | bobbyhadz*. [Online; accessed 25. Oct. 2023]. Feb. 2023. URL: https://bobbyhadz.com/blog/python-error-invalid-command-bdist-wheel.

[8] *Re: [casper] Cannot write to certain registers*. [Online; accessed 25. Oct. 2023]. Oct. 2023. URL: https://www.mail-archive.com/casper@lists.berkeley.edu/msg02503.html.

[9] *Red-Pitaya-Casper-Toolflow*. [Online; accessed 25. Oct. 2023]. Oct. 2023. URL: https://github.com/wintervolcano/Red-Pitaya-Casper-Toolflow.git.

[10] *Snapshot — CASPER Toolflow 0.1 documentation*. [Online; accessed 25. Oct. 2023]. Mar. 2023. URL: https://casper-toolflow.readthedocs.io/en/latest/src/blockdocs/Snapshot.html.